Bluetooth Low Energy: Programação

Tecnologias de Comunicação para a Internet das Coisas, Aula 05

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

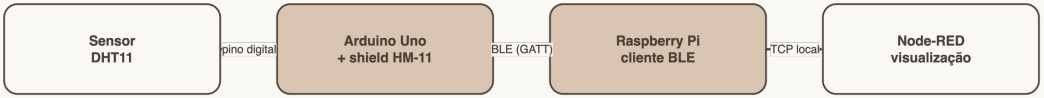
Agenda (1/2)

- Desenvolvimento de aplicações BLE: arquitetura do exercício
- Bibliotecas e wrappers disponíveis: BlueZ, bluepy, PyGATT e SimpleBLE
- A biblioteca bluepy: modelo de classes
- Implementação de um varredor (*scanner*)
- Trabalho com notificações

Agenda (2/2)

- O servidor BLE em Arduino: shield HM-11 e comunicação série
- Verificação do lado do gateway

Arquitetura da Cadeia Completa



Arquitetura do Exercício (1/2)

- O objetivo desta aula é implementar, na prática, os conceitos de GAP e GATT vistos na aula anterior.
- Do lado do sensor, um Arduino equipado com um shield BLE assume o papel de periférico, ou seja, de servidor GATT.
- Do lado do gateway, um computador com Linux (nas aulas seguintes, um Raspberry Pi) assume o papel de central, ou seja, de cliente GATT.

Arquitetura do Exercício (2/2)

- O cliente descobre o periférico, estabelece ligação, e lê ou escreve características para obter as leituras dos sensores.
- Esta divisão de papéis reproduz a arquitetura típica de uma solução de IoT: um nó restrito que recolhe dados e um nó mais capaz que os agrega e encaminha.

Bibliotecas e Wrappers para BLE (1/2)

- **BlueZ:** pilha protocolar Bluetooth oficial do Linux. Suporta todos os protocolos e camadas centrais do Bluetooth. Os pacotes `bluez-utils` e `bluez-firmware` fornecem os utilitários de baixo nível e o firmware necessário.
- **bluepy:** interface Python para BLE, construída sobre o BlueZ. É a biblioteca usada nesta unidade curricular.
- **PyGATT:** alternativa em Python, com interface centrada nas operações GATT.

Bibliotecas e Wrappers para BLE (2/2)

- **SimpleBLE:** biblioteca com API simplificada, orientada a casos de utilização comuns.
- A escolha da biblioteca condiciona o nível de abstração disponível, mas todas assentam, em Linux, sobre a mesma pilha BlueZ.

bluepy: Modelo de Classes (1/2)

- **Peripheral:** representa um dispositivo periférico ao qual o programa se liga. É o ponto de entrada para todas as operações sobre um dispositivo concreto.
- **Scanner:** procura dispositivos BLE que estejam a anunciar-se na vizinhança.
- **ScanEntry:** representa um dispositivo encontrado durante o varrimento, com o respetivo endereço e dados de anúncio.
- **DefaultDelegate:** classe base cujos métodos são invocados quando ocorrem eventos assíncronos, como a descoberta de um dispositivo ou a chegada de uma notificação.
- **UUID:** representa os identificadores de serviços e características.

bluepy: Modelo de Classes (2/2)

- **Service, Characteristic e Descriptor:** correspondem diretamente à hierarquia GATT estudada na aula anterior.
- O mapeamento entre estas classes e os conceitos GATT é direto, o que facilita a passagem da teoria à implementação.

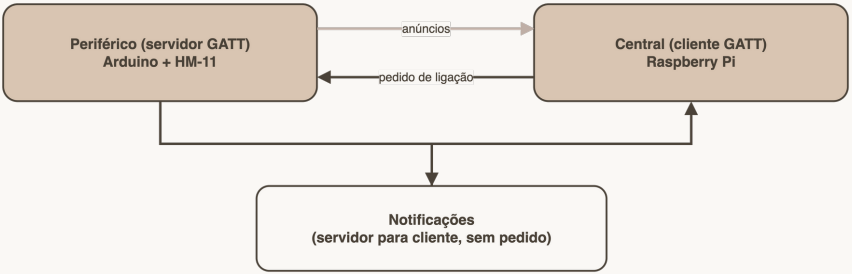
Implementação de um Varredor (1/2)

- O varrimento é o primeiro passo de qualquer aplicação cliente: sem descobrir o periférico, não há ligação possível.
- Cria-se uma subclasse de `DefaultDelegate` e redefine-se o método `handleDiscovery`, que recebe o dispositivo encontrado e dois indicadores: se é um dispositivo novo e se traz dados novos.
- Instancia-se um `Scanner` e associa-se-lhe o delegado através do método `withDelegate`.
- Invoca-se `scan` com a duração pretendida, em segundos, e obtém-se a lista de dispositivos encontrados.

Implementação de um Varredor (2/2)

- Para cada dispositivo, ficam disponíveis o endereço (`addr`), o tipo de endereço e a potência do sinal recebido (RSSI), em dBm.
- O valor de RSSI é útil para estimar proximidade, embora não constitua uma medida fiável de distância.

Papéis e Sentido das Trocas



Notificações: Conceito (1/2)

- Ler periodicamente uma característica obriga o cliente a interrogar o servidor de forma repetida, mesmo quando não há dados novos, abordagem simples, mas ineficiente em consumo.
- As notificações invertem a iniciativa: é o servidor que informa o cliente sempre que o valor de uma característica se altera.
- No protocolo, distinguem-se notificações e indicações. A notificação não exige confirmação do cliente; a indicação exige, o que a torna mais fiável e mais dispendiosa.

Notificações: Conceito (2/2)

- Este mecanismo é particularmente adequado a sensores que reportam eventos esporádicos, como a deteção de movimento ou a ultrapassagem de um limiar.
- É também o mecanismo que sustenta a integração com um broker de mensagens, tratada mais adiante na unidade curricular.

Operações GATT e Custo Energético



O cliente pede, o servidor responde

O cliente envia um valor

O servidor envia sem ser pedido

Sondagem periódica consome energia; a notificação só transmite quando há novidade

Notificações: Implementação em bluepy (1/2)

- Define-se uma subclasse de `DefaultDelegate` e redefine-se o método `handleNotification`, que recebe dois argumentos: o *handle* da característica que gerou a notificação e os dados recebidos.
- O *handle* permite distinguir a origem quando várias características estão configuradas para notificar.
- Cria-se o objeto `Peripheral` com o endereço do dispositivo e associa-se o delegado através de `setDelegate`.
- Obtém-se o serviço pretendido com `getServiceByUUID` e, a partir dele, a característica com `getCharacteristics`.

Notificações: Implementação em bluepy (2/2)

- Ativa-se a notificação escrevendo no descritor de configuração da característica.
- O programa entra depois num ciclo de espera, no qual chama `waitForNotifications`; sempre que chega uma notificação, o método `handleNotification` é invocado automaticamente.

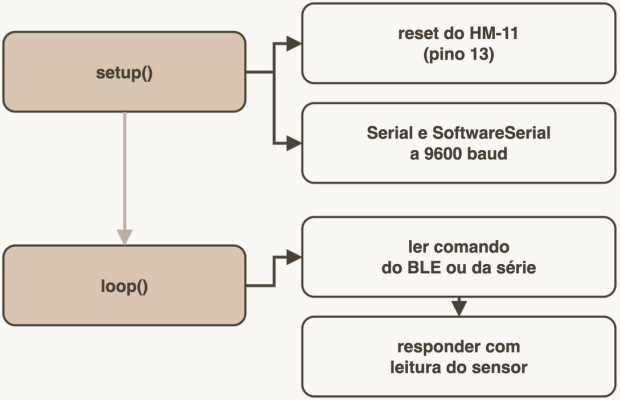
O Shield BLE HM-11 (1/2)

- O exercício prático recorre a um shield de comunicações BLE instalado sobre o Arduino, baseado no módulo HM-11.
- A configuração do modo de funcionamento faz-se através da colocação de *jumpers* no shield, antes da montagem sobre a placa.
- O módulo comunica com o Arduino por porta série, o que exige a definição de uma porta série por software, uma vez que a porta série física está ocupada pela ligação ao computador.

O Shield BLE HM-11 (2/2)

- O pino de *reset* do módulo é controlado por uma saída digital do Arduino, o que permite reinicializar o módulo no arranque do programa.
- Ambas as portas série (a física, para o monitor série, e a virtual, para o módulo BLE) operam a 9600 bit/s.

Estrutura do Sketch Arduino



Estrutura do Programa Arduino (1/2)

- Na função `setup`, o pino de *reset* é configurado como saída e o módulo é reinicializado através de um impulso, seguido da inicialização das duas portas série.
- Na função `loop`, o programa verifica se há dados disponíveis vindos do módulo BLE ou do monitor série, e lê o carácter recebido.
- O carácter recebido funciona como comando: `t` para temperatura, `h` para humidade, `l` para luminosidade e `X` para o conjunto de todas as grandezas.

Estrutura do Programa Arduino (2/2)

- A cada comando corresponde uma resposta enviada simultaneamente para o módulo BLE e para o monitor série, o que facilita o diagnóstico durante o desenvolvimento.
- Cada comando define ainda um fator de escala aplicado ao atraso do ciclo, o que permite ajustar a periodicidade de resposta consoante a grandeza pedida.

Verificação do Lado do Gateway (1/2)

- O comando `hcitool dev` lista os adaptadores Bluetooth disponíveis no sistema, com os respetivos identificadores e endereços físicos.
- O comando `hcitool lescan` inicia o varrimento de dispositivos BLE na vizinhança, apresentando os endereços encontrados e, quando disponível, o nome anunciado pelo dispositivo.
- Identificado o endereço do periférico, estabelece-se a ligação com `gatttool -b <endereço> -I`, que abre a sessão em modo interativo.
- Dentro dessa sessão, o comando `connect` estabelece a ligação efetiva ao dispositivo.

Verificação do Lado do Gateway (2/2)

- Segue-se a descoberta dos serviços e características expostos, e a leitura ou escrita dos respetivos valores.
- Esta verificação por linha de comandos deve preceder o desenvolvimento em Python: confirma que a ligação é possível antes de se introduzir a complexidade do código.

Método de Trabalho Recomendado (1/2)

- Preparar e testar o hardware isoladamente: verificar o funcionamento do sketch através do monitor série, antes de envolver a comunicação BLE.
- Confirmar a visibilidade do periférico com `hcitool lescan`, garantindo que o módulo está efetivamente a anunciar-se.
- Estabelecer a ligação manualmente com `gatttool` e explorar os serviços e características disponíveis.

Método de Trabalho Recomendado (2/2)

- Só depois passar ao desenvolvimento em Python, começando pelo varrimento e progredindo para a leitura de características e para as notificações.
- Esta progressão isola a origem dos problemas: uma falha detetada no passo três não pode ser atribuída ao código Python, que ainda não foi escrito.

Síntese da Aula (1/2)

- A programação BLE traduz diretamente a hierarquia GATT: serviços, características e descritores têm correspondência nas classes das bibliotecas usadas.
- Em Linux, a pilha BlueZ é a base sobre a qual assentam as bibliotecas de mais alto nível, entre as quais a bluepy usada nesta unidade curricular.
- O padrão de delegação (subclasse de `DefaultDelegate` com redefinição de `handleDiscovery` ou `handleNotification`) é o mecanismo através do qual a aplicação reage a eventos assíncronos.

Síntese da Aula (2/2)

- As notificações substituem a interrogação periódica, reduzindo o consumo e a latência, e são o mecanismo natural para encaminhar leituras para um middleware de mensagens.
- Nas aulas seguintes, o papel de cliente passa para um Raspberry Pi, que atuará como gateway entre a rede BLE e a rede IP.

Fontes

Gonçalves, P. *Apresent5 — BLE Dev* e *guia 4 — arduinoBLE*, TCIoT, edições anteriores. Base do modelo de classes bluepy, do varredor, das notificações e do exercício com o shield HM-11.

Bluetooth SIG. *Bluetooth Core Specification*. Definição das operações GATT de leitura, escrita, notificação e indicação.

Documentação da biblioteca bluepy: `ianharvey.github.io/bluepy-doc/`

Obrigada.

Questões e comentários